

Nettoyage des données

Analyse des données — Séance 3

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

Où nous allons aujourd'hui

1. **Reprise** : sélectionner, filtrer, vérifier. Vingt-cinq minutes.
2. Les valeurs manquantes : les trouver, et comprendre d'où elles viennent.
3. Décider quoi en faire — et assumer la décision.
4. Les doublons, qui ne sont pas tous des erreurs.

La première partie referme la séance précédente. La suite ouvre le vrai sujet.

Deux fichiers aujourd'hui : `cm02-communes.csv`, que vous connaissez, puis `cm03-salaires.csv`, que nous ouvrons ensemble.

Reprise — ce qu'on n'a pas eu
le temps de voir

Choisir des colonnes

```
df["POP"]                                # une colonne (Series)
df[["LIBGEO", "POP", "TAUX_CHOMAGE"]]    # plusieurs (DataFrame)
```

Les doubles crochets ne sont pas une coquetterie : vous passez une *liste* de noms. C'est la structure vue en séance 1.

```
df["CAT_URBAINE"].value_counts()
df["CAT_URBAINE"].value_counts(normalize=True)
```

`value_counts()` est le premier réflexe sur toute variable qualitative. Les quatre modalités affichées doivent correspondre à celles du dictionnaire — sinon, quelque chose ne va pas.

Filtrer des lignes

```
df[df["POP"] > 100000]
```

La condition à l'intérieur produit une suite de vrai/faux, un par ligne. Le tableau ne conserve que les lignes vraies.

```
df[(df["POP"] > 50000) & (df["TAUX_CHOMAGE"] > 18)]      # et  
df[(df["CAT_URBAINE"] == "1") | (df["CAT_URBAINE"] == "2")]  # ou  
df[df["CODGEO"].str[:2].isin(["75", "93", "95"])]          # appartenance
```

Les parenthèses autour de chaque condition sont obligatoires. Sans elles, Python évalue dans le mauvais ordre et renvoie une erreur peu explicite.

Trier pour repérer une anomalie

```
df.sort_values("NB_ENTREPRISES", ascending=False).head(15)
```

ascending=False trie du plus grand au plus petit. Douze communes dépassent 9 999 établissements : les deux valeurs suspectes n'apparaissent qu'aux rangs 13 et 14.

Un effectif brut classe surtout les grandes villes. Rapportez-le à la population.

```
(df.assign(par_hab=df["NB_ENTREPRISES"] / df["POP"])\n     .sort_values("par_hab", ascending=False).head(5))
```

Saint-Junien et Douai passent en tête : 9 999 établissements pour 11 245 et 39 508 habitants, soit 0,89 et 0,25 par personne. C'est un code de non-réponse, pas un effectif.

Compter, avant et après

```
len(df)                                # 45 lignes
df["TAUX_CHOMAGE"].isna().sum()        # 6 valeurs manquantes

grandes = df[df["POP"] > 100000]
print(len(df), "->", len(grandes))    # 45 -> 16
```

Comptez avant et après chaque filtrage, systématiquement. Un filtre qui supprime les trois quarts de l'échantillon signale généralement une erreur de condition, pas un fait économique.

Ici la perte est voulue : seize communes de plus de cent mille habitants sur quarante-cinq. Mais vous ne le savez que parce que vous avez compté.

Trois vérifications avant de quitter le fichier

1. `df.shape` correspond-il à ce qu'annonce la documentation ?
2. `df.info()` : chaque colonne a-t-elle le type attendu ?
3. `value_counts()` sur les variables qualitatives : les modalités correspondent-elles au dictionnaire ?

Ces trois contrôles prennent deux minutes. Ils vous éviteront la moitié des erreurs du semestre — et ils constituent une question type du contrôle continu.

Vous savez charger un fichier, comprendre ce qu'il contient, en extraire ce qui vous intéresse et vérifier que vous ne vous êtes pas trompée.

Il reste le travail que personne ne montre : **rendre ces données utilisables**.

Le principe de la suite

Le nettoyage n'est pas une opération technique préalable à l'analyse. C'est une suite de **décisions**, dont chacune doit pouvoir se justifier — et dont chacune modifie le résultat.

Les valeurs manquantes

Les trouver

```
df.isna().sum()           # effectif par colonne
df.isna().mean().round(3)  # part par colonne
```

Sur le fichier des communes, une fois les conversions de la séance 2 faites :

Colonne	Manquants	Part
TAUX_CHOMAGE	6	13,3 %
NB_ENTREPRISES	2	4,4 %
Les cinq autres	0	0 %

Deux colonnes seulement, et sur quarante-cinq lignes. C'est un fichier de cours. La question qui suit, elle, est la même sur tous les fichiers.

La question qui compte : *qui* sont les manquants ?

```
df.groupby("CAT_URBAINE")["TAUX_CHOMAGE"].apply(  
    lambda s: s.isna().sum())
```

Catégorie urbaine	Communes	Sans taux	Part
1 — grand pôle	13	0	0 %
2 — couronne	16	0	0 %
3 — petit pôle	10	0	0 %
4 — rural	6	6	100 %

Les six valeurs absentes sont les six communes rurales, et elles seulement.

L'absence n'est pas un accident : elle est produite par le secret statistique, qui dépend de la taille de la commune. Une variable que vous observez explique parfaitement ce qui manque.

Les manquants qui ne ressemblent pas à des manquants

Codage	Exemple	Symptôme
Code numérique	9, 99, 999, 9 999	Un effectif d'entreprises trop rond
Texte	"nd", "NR", "-"	Colonne numérique lue en texte
Chaîne vide	" "	Comptée comme une modalité
Valeur impossible	-1, 0, 1900	Aucun symptôme

Vous avez rencontré les deux premiers en séance 2 : "nd" dans TAUX_CHOMAGE, et 9 999 dans NB_ENTREPRISES.

Pandas ne les reconnaît pas : `isna()` renvoie zéro manquant, et tous les calculs fonctionnent.

Seul le dictionnaire des variables vous dira comment la non-réponse est codée.

Un second fichier

Quarante-cinq communes ne suffisent pas pour la suite. Nous passons à des données individuelles.

```
brut = pd.read_csv("cm03-salaires.csv",
                  encoding="latin-1", sep=";", decimal=".", thousands=" ",
                  dtype={"IDENT": str, "SECTEUR": str, "REGION": str})
df = brut.copy()
print(brut.shape)           # (8012, 15)
```

Huit mille douze lignes, quinze colonnes. **Une ligne est un poste occupé dans l'année**, pas une personne : retenez-le, nous y reviendrons.

Données fabriquées pour l'enseignement. Ordres de grandeur plausibles, valeurs fictives. Le dictionnaire est sur le site du cours.

Le calcul qui fonctionne, et qui est faux

```
df["SALAIRE_NET"].mean()          # 67745.36
```

Soixante-sept mille euros par mois. Aucune erreur, aucun avertissement. Regardons la colonne au lieu de la moyenner :

```
df["SALAIRE_NET"].value_counts().head(4)
```

Valeur	Effectif	Ce que dit le dictionnaire
0	675	Sans objet : non-salariés
999 999	525	Non-réponse
10 000	72	Plafond de diffusion

Trois problèmes différents dans une seule colonne. Aucun des trois ne se traite comme les autres.

Trois origines, trois conséquences

Manquant complètement au hasard

L'absence ne dépend de rien. Un questionnaire égaré, une ligne corrompue.

Supprimer les lignes réduit la précision, mais ne biaise pas.

Manquant au hasard, conditionnellement

L'absence dépend de variables *observées*. Le taux de chômage manque pour les communes rurales, et vous observez la catégorie urbaine.

Supprimer biaise ; on peut corriger en tenant compte de la variable en cause.

Manquant non au hasard

L'absence dépend de la valeur *elle-même*. Les hauts revenus ne déclarent pas leur revenu.

Aucune correction n'est possible à partir des seules données observées.

Reconnaître le troisième cas

```
df["nr"] = df["SALAIRE_NET"] == 999999  
100 * df.groupby("DIPLOME")["nr"].mean()
```

Diplôme	aucun	BEPC	CAP	bac	bac+2	bac+3/4	bac+5
Non-réponse	3,3 %	4,4 %	4,9 %	6,3 %	8,0 %	7,7 %	10,6 %

Le taux de non-réponse est trois fois plus élevé en haut de l'échelle des diplômes qu'en bas.

Ce tableau ne prouve pas que la non-réponse dépend du salaire lui-même. Il montre qu'elle dépend d'une variable qui lui est fortement liée, ce qui rend l'hypothèse de hasard intenable.

C'est tout ce que les données peuvent dire. Le reste est un raisonnement, pas une mesure.

Sur les variables de revenu et de patrimoine, la non-réponse est presque toujours du troisième type.

- Les très hauts revenus déclarent moins souvent.
- Les revenus non déclarés sont donc, en moyenne, plus élevés.
- Supprimer ces lignes sous-estime la moyenne *et* les inégalités.

On ne peut pas trancher à partir des données seules : par construction, on ne voit pas ce qui manque.

Ce qu'on peut faire, c'est **le dire**, et mesurer la sensibilité du résultat à l'hypothèse retenue.

Hors champ n'est pas non-réponse

Trois colonnes du fichier contiennent des cases vides. Aucune des trois ne signifie la même chose.

Colonne	Effectif	Ce que le vide signifie
SALAIRE_NET = 0	675	Non-salarié : la question ne se pose pas
MOTIF_ABS vide	7 153	N'a pas été absent : sans objet
PRIME vide	4 442	N'a touché aucune prime

Une personne **hors champ** n'aurait pas dû répondre. Une **non-réponse** aurait dû. Les traiter de la même façon est une faute.

Remplacer une prime absente par la prime moyenne invente un versement qui n'a pas eu lieu. Supprimer la ligne élimine précisément les salariés sans prime, c'est-à-dire la majorité.

La modalité 9 n'est pas toujours une non-réponse

Code	MOTIF_ABS — libellé du dictionnaire	Effectif
1	Congés	458
2	Maladie	175
3	Congé de maternité ou de paternité	34
4	Formation	54
5	Raisons familiales	45
9	Chômage partiel	61
99	Non-réponse	32

Ici la non-réponse est codée **99**, et **9** est une vraie situation.

`df["MOTIF_ABS"].replace(9, np.nan)` détruirait soixante et une situations de chômage **partiel** : sans message d'erreur, et en supprimant exactement la catégorie qu'un économiste du travail cherchait.

La censure haute

Une valeur qui revient beaucoup trop souvent dans une variable continue signale un plafond.

```
df["HEURES"].value_counts().sort_index().tail(8)
```

Heures	54	55	56	57	58	59	60
Effectif	35	32	35	29	27	35	273

Le dictionnaire tranche : la modalité se lit « 60 heures ou plus ». Ce n'est pas une erreur, c'est un regroupement décidé par le producteur. Même chose sur SALAIRE_NET, où les quantiles à 99 % et à 99,9 % valent tous deux 10 000 euros.

Un indice de Gini calculé sur cette colonne est une borne inférieure. La censure comprime le haut de la distribution, là où se concentre l'inégalité mesurée.

Décider

Trois options, aucune neutre

Option	Ce qu'on fait	Ce que cela coûte
Supprimer les lignes	on retire les observations incomplètes	échantillon réduit, biais possible
Imputer	on remplace par une valeur	fausse précision
Conserver et signaler	on ne touche à rien, on rapporte les effectifs	analyse plus lourde

Signaler n'est pas une action sur les données : c'est une action sur le compte rendu. On laisse les manquants en place, les calculs les ignorent, et on dit combien d'observations ont réellement servi.

C'est le cas général — et la seule option qui ne suppose rien. Supprimer suppose que les manquants sont sans structure ; imputer suppose qu'on sait par quoi les remplacer.

Supprimer : ce qui disparaît vraiment

```
avant = len(com)
propre = com.dropna(subset=["TAUX_CHOMAGE", "NB_ENTREPRISES"])
print(avant, "->", len(propre))           # 45 -> 37
```

Huit communes sur quarante-cinq, soit 18 % du fichier. Le vrai problème n'est pas le nombre :

	Conservées	Supprimées
Population moyenne	156 380	1 786
Communes rurales	0	6

Le contrôle à faire

Comparez les lignes supprimées aux lignes conservées. Ici la catégorie rurale disparaît entièrement : le fichier nettoyé ne décrit plus les communes françaises, mais celles de plus de trois mille habitants.

Le piège de l'indexation chaînée

```
com[com["POP"] < 2000]["TAUX_CHOMAGE"] = 0
```

SettingWithCopyWarning: A value **is** trying to be **set** on
a copy of a **slice from** a DataFrame

Deux crochets successifs produisent une *copie* : la modification s'applique à la copie, puis disparaît. Aucune erreur, aucun effet.

```
com.loc[com["POP"] < 2000, "TAUX_CHOMAGE"] = 0
```

`.loc` désigne lignes et colonnes en une seule opération. **Pour modifier, c'est toujours `.loc`.**

Chaîner plutôt qu'empiler

```
# Empile des variables intermediaires
df1 = df[df["STATUT"] != 3]
df2 = df1[["IDENT", "DIPLOME", "SALAIRE_NET"]]
df3 = df2.sort_values("SALAIRE_NET", ascending=False)

# Chaîne : une seule expression, lisible de haut en bas
resultat = (
    df
    .loc[df["STATUT"] != 3,
         ["IDENT", "DIPLOME", "SALAIRE_NET"]]
    .sort_values("SALAIRE_NET", ascending=False)
)
```

Les parenthèses extérieures autorisent le retour à la ligne. C'est l'idiome courant de pandas : une opération par ligne, dans l'ordre où on les pense.

Supprimer une ligne, ou supprimer une colonne

```
df.dropna(subset=["SALAIRE_NET"])    # retire des OBSERVATIONS  
df.drop(columns=["PRIME"])           # retire une VARIABLE
```

Deux opérations, deux pertes de nature différente.

- Retirer des **lignes** réduit l'échantillon, et le déforme si les manquants ne sont pas au hasard.
- Retirer une **colonne** conserve tout l'échantillon, mais vous prive d'une variable.

Ici PRIME est vide pour 55 % des postes, et pourtant le vide y est une information. SALAIRE_NET manque pour 7 % des salariés, ce qui ne justifie pas de perdre 7 % de l'échantillon si votre question porte sur la durée du travail.

La question n'est pas « combien manque-t-il ? » mais « en ai-je besoin ? ».

Regarder avant de décider

Aucun tableau de nombres ne remplace un coup d'œil sur la distribution.

```
df["SALAIRE_NET"].hist(bins=60)           # la forme d'ensemble  
df.boxplot(column="SALAIRE_NET", by="DIPLOME") # les points isolés  
df.plot.scatter(x="AGE", y="SALAIRE_NET")    # les incohérences
```

- L'**histogramme** montre ici deux anomalies visibles à l'œil : la barre à zéro, et la barre au plafond de 10 000.
- La **boîte à moustaches** isole les points éloignés, sans dire s'ils sont faux.
- Le **nuage de points** montre ceux qui contredisent la logique économique.

Nous reviendrons sur les graphiques en séance 6. Ici ils servent à **diagnostiquer**, pas à communiquer.

Imputer par la moyenne : ce que cela casse

```
df["SALAIRE_NET"].fillna(df["SALAIRE_NET"].mean())
```

Sur les 7 326 postes salariés, dont 524 sans salaire renseigné :

	Conserver	Imputer
Moyenne	2 610	2 610
Écart-type	1 465	1 412
Corrélation avec le diplôme	0,356	0,342
Observations annoncées	6 802	7 326

La moyenne ne bouge pas d'un centime. C'est ce qui rend l'opération dangereuse : rien ne signale que quelque chose a changé.

Ce qui change, c'est la dispersion, les corrélations, et le nombre d'observations annoncé. Les erreurs-types seront trop petites, et l'inférence trop confiante.

Quand l'absence est une information

```
connu = sal.loc[sal["PRIME"] != 999999, "PRIME"]    # sans les NR
connu.mean()                                       # sur les primes réellement percues
connu.fillna(0).mean()                           # en comptant les absences comme zero
```

Décision	Prime moyenne	Observations
Moyenne des primes versées	1 376	3 229
Prime moyenne par salarié	633	7 022

Les deux chiffres sont justes. Ils ne répondent pas à la même question, et l'écart est d'un facteur deux.

Seul le dictionnaire tranche entre « non renseigné » et « sans objet ». Le premier se laisse manquer, le second se code zéro.

1. **Conserver et signaler** : ne rien retirer, et rapporter le nombre d'observations effectivement utilisées pour chaque statistique. C'est la colonne « Observations » de tout tableau d'article.
2. **Documenter** : dire combien de lignes sont concernées et sur quelles variables.
3. **Tester la sensibilité** : refaire le calcul principal sous deux hypothèses, et regarder si la conclusion tient.

Un résultat qui change de signe selon le traitement des manquants n'est pas un résultat. Un résultat qui tient sous les deux hypothèses est solide, et le dire renforce l'argument.

Écrire votre première fonction

Quatre colonnes de ce fichier codent la non-réponse par un nombre. La règle s'écrit une fois.

```
def nettoyer(serie, codes_manquants):  
    s = pd.to_numeric(serie, errors="coerce")  
    return s.replace(codes_manquants, np.nan)  
  
df["SALAIRE_NET"] = nettoyer(df["SALAIRE_NET"], [999999])  
df["PRIME"]       = nettoyer(df["PRIME"],       [999999])  
df["HEURES"]      = nettoyer(df["HEURES"],      [99])  
df["DIPLOME"]     = nettoyer(df["DIPLOME"],     [9])
```

Trois bénéfices : la règle est écrite une seule fois, elle porte un nom qui dit ce qu'elle fait, et la corriger la corrige partout.

Notez la ligne absente : MOTIF_ABS. La fonction est correcte ; la liste de codes ne l'est jamais par défaut. Elle vient du dictionnaire, colonne par colonne.

Les doublons

Deux questions différentes

```
df.duplicated().sum()          # 12  
df.duplicated(subset=["IDENT"]).sum()  # 632
```

- Douze lignes sont **entièrement identiques** à une autre. C'est presque toujours une erreur : import répété, fusion mal faite.
- Six cent trente-deux **identifiants sont répétés**. Ce n'est pas forcément une erreur. Tout dépend de l'unité statistique.

Rappel de la diapositive d'ouverture : **une ligne est un poste**, pas une personne. Une même personne peut occuper deux postes dans l'année.

La clé du fichier n'est pas IDENT, c'est le couple IDENT + NOPOSTE.

Regarder avant de supprimer

```
doublons = df[df.duplicated(subset=["IDENT"], keep=False)]  
doublons.sort_values(["IDENT", "NOPOSTE"]).head(6)
```

IDENT	NOPOSTE	TEMPS	HEURES	SALAIRE_NET
00006195	1	1	34	2 499,21
00006195	2	2	27	560,61

Les deux lignes diffèrent sur le temps de travail, les heures et le salaire. Ce sont deux emplois distincts, pas une erreur.

Le diagnostic, et l'ordre des opérations

On supprime d'abord les douze lignes identiques partout. Il reste 8 000 lignes et 620 identifiants répétés : ceux-là se gardent. Si votre question porte sur les personnes, il faut agréger, pas supprimer.

Documenter

Ne jamais écraser les données brutes

```
brut = pd.read_csv("cm03-salaires.csv", sep=";")    # jamais modifie  
df    = brut.copy()                                # on travaille ici
```

Deux raisons.

- Vous vous tromperez, et il faudra revenir en arrière sans tout retélécharger.
- La comparaison brut / nettoyé est ce qui vous permet de *mesurer* l'effet de vos décisions.

Le journal de nettoyage

Étape	Décision	Lignes	Salaire moyen	Justification
Chargement	—	8 012	67 745	Fichier brut
Doublons techniques	Suppression	8 000	67 719	Lignes identiques partout
Code 999 999 → manquant	Conversion	8 000	2 375	Dictionnaire, SALAIRE_NET
Non-salariés	Exclusion	7 326	2 610	Hors champ : pas de salaire

Trois observations sur ce tableau.

- Supprimer douze lignes sur huit mille ne change presque rien : 67 745 devient 67 719.
- Traiter un seul code de non-réponse fait passer la moyenne de 67 745 à 2 375. Une ligne de code, un facteur vingt-huit.
- Retirer les non-salariés la fait remonter à 2 610, parce que leurs zéros tiraient la moyenne vers le bas.

Une décision non documentée est une décision cachée.

Ce qu'un article ne montre pas

Tout article empirique commence par un tableau de statistiques descriptives. Voici le nôtre.

	Moyenne	Observations
Salaire mensuel net	2 610	6 802
Âge	42,3	7 326
Heures hebdomadaires	34,0	7 089
Part de diplômés du supérieur	0,42	7 033

- Pourquoi chaque ligne compte-t-elle un nombre différent d'observations ?
- Où sont passées les 686 lignes du fichier initial ?
- Les salaires nuls ont-ils été conservés ?

Aucune réponse n'est dans le tableau. Toutes sont dans la note méthodologique — si elle existe.

Trois questions avant de passer à l'analyse

1. Combien de lignes ai-je perdues, et en quoi diffèrent-elles de celles que je garde ?
2. Chacune de mes décisions est-elle justifiable à voix haute devant quelqu'un qui conteste ?
3. Mon résultat principal survivrait-il à la décision inverse ?

La troisième est celle qui distingue un travail solide d'un travail plausible. C'est aussi une question type du contrôle continu.

Pour la suite

- Repérer les non-réponses codées en chiffres, et les convertir
- Distinguer non-réponse et hors champ — deux choses différentes
- Identifier une censure haute et dire ce qu'elle fait à un indice d'inégalité
- Documenter chaque décision, et mesurer son effet sur le résultat

Le livrable n'est pas un fichier propre : c'est le **journal des décisions**, avec l'effet chiffré de chacune.

1. Terminez votre journal de nettoyage sur `cm03-salaires.csv`.
2. Recalculez le salaire moyen après chaque étape et notez l'écart.
3. Reprenez `MOTIF_ABS` et `CONTRAT` : quels codes sont des non-réponses, lesquels sont du hors champ ?
4. Vérifiez que le notebook s'exécute de haut en bas.

`stefaniamarcassa.github.io/analyse_des_donnees`

Des questions ?